

Cyber-Alert Prioritization Using ML and Network Analytics

Sajida Al Romana
Computer Science Department
Kean University
New Jersey, United States

Abstract—In enterprise Security Operations Centers (SOCs), the large number of alerts generated daily can cause ‘alert fatigue’ among security analysts and make it harder to detect critical network intrusions. Static rule-based filtering can struggle to separate high-risk attacks from routine network anomalies in diverse traffic. This research builds a data-driven approach for prioritizing cyber alerts using network traffic features. Using the NSL-KDD benchmark dataset, a Random Forest classifier is trained to assign records to three priority levels: Low, Medium, and High. The proposed model achieves a test accuracy of 74.80% on the official KDDTest+ set, with precision and recall varying notably across classes; only 57% of High-priority alerts were recognized as High, and 42% were labeled Low. In a feature-ablation test, a model retrained without four of the five most important features (src_bytes, dst_bytes, dst_host_error_rate, diff_srv_rate) reached 74.19% accuracy. Adding small Gaussian noise ($\mu = 0$, $\sigma = 0.1$) to seven byte- and count-valued features left accuracy unchanged at 74.80%; because $\sigma = 0.1$ is tiny relative to those features’ scales, this shows only insensitivity to very small perturbations. SHAP feature attributions and an example decision tree were used to show which features the model relies on. This is a small-scale study on a static benchmark dataset and does not evaluate live traffic.

I. INTRODUCTION

Enterprise networks generate large volumes of security alerts every day, and separating real intrusions — such as DDoS floods or targeted network attacks — from routine noise is a persistent operational challenge for SOC teams. To protect organizational digital infrastructure from these threats, modern security teams primarily rely on Intrusion Detection Systems (IDS) and Security Information and Event Management (SIEM) platforms. However, a significant limitation of these systems is that they log even minor network deviations and generate thousands of security alerts daily. SOC practitioners commonly report that many of these alerts are routine network anomalies or “false positives” rather than attacks.

Dealing with so many irrelevant notifications can lead to “alert fatigue” among security analysts. When analysts must manually check thousands of routine alerts every day, their attention and productivity can decline. As a result, real high-risk incidents such as intrusions or data theft may go unnoticed for long periods, which can seriously harm the organization.

Many alert filtering setups still depend on static, rule-based configurations. These can be slow to adapt to new attack tactics and make limited use of traffic behavior when ranking alerts. This suggests room for a data-driven approach. A model that

learns from traffic features could help separate likely attacks from ordinary network noise.

This study builds a machine learning model that assigns network connection records to Low, Medium, or High priority. Rather than reporting accuracy alone, this study also checks how the model behaves under feature removal and small input noise, and inspects its decisions with SHAP. The study includes a feature-ablation test, a Gaussian noise test, and SHAP-based explanations, and asks how far the model’s behavior holds up beyond a clean benchmark evaluation.

II. RELATED WORK

Many studies have used the NSL-KDD dataset [1] for network intrusion detection. NSL-KDD is a simulated benchmark, so results on it may not carry over to real SOC traffic. Alert fatigue and the challenge of prioritizing security alerts have also been studied more broadly in the SOC operations literature. More recent work has applied SHAP specifically to improve model interpretability in intrusion detection systems [2], [3]; Gaspar et al. [3] additionally validated their explanations through a perturbation analysis on the top identified features, though neither study evaluates robustness under a Gaussian noise stress test as performed in this work. In the studies we reviewed, two gaps stood out:

First, many frameworks use models such as Random Forest or Extreme Gradient Boosting (XGBoost) as black boxes, giving a prediction without a clear reason for why a connection received a particular risk level. Analysts are unlikely to trust automated rankings without some explanation.

Second, many NSL-KDD studies report results on clean benchmark data without checking how the model behaves when features are removed or perturbed.

This study tries to address both gaps on a small scale. We compare three tree-based classifiers (Decision Tree, Random Forest, XGBoost) on the same train/test split. We use SHAP to show global feature contributions, and we also check the model’s behavior under partial feature removal and injected noise.

III. PROPOSED METHODOLOGY

The goal of the pipeline is to take network connection records and assign each one a risk level. This pipeline has four main stages: dataset organization, categorical data transformation, ensemble-based model training, and priority mapping.

A. Dataset Selection and Suitability Analysis

This study uses the benchmark NSL-KDD dataset [1]. Unlike its predecessor, the KDD Cup 99 dataset, NSL-KDD removes exact duplicate records; the Limitations section notes that many near-duplicate records may remain. The dataset contains normal traffic and several attack categories. To reduce the risk of data leakage, this study uses the official NSL-KDD partition: KDDTrain+ (125,973 records) for model training and KDDTest+ (22,544 records) as a fully independent, held-out evaluation set.



Fig. 1. Overview of the alert prioritization pipeline.

B. Feature Encoding

Network logs contain mixed types of data or features. Our dataset comprises a total of 41 initial features, including continuous numerical profiles (such as traffic duration, `src_bytes`, `dst_bytes`, etc.) and categorical or text-based protocol attributes (such as `protocol_type`, `service`, and `flag`). Before being fed into the machine learning algorithm, text-based values like protocol type, service, and flag were converted into a machine-readable numerical format using pandas `get_dummies` one-hot encoding (with `drop_first=True`). This encoding process expanded our feature space from the original 41 columns to 119 encoded features, so the tree-based models can use these attributes. Train and test sets were one-hot encoded separately, and test-set columns were aligned to the training feature set (missing categories filled with zero, and test-only categories dropped) to prevent any information from the test set influencing the encoding.

C. Random Forest Ensemble and Threat Priority Mapping

A Random Forest Classifier [4] serves as the core engine of this predictive system. Random Forest was used because it is a common baseline for tabular data and works with SHAP’s TreeExplainer. It is often described as resistant to overfitting, but the training-versus-test accuracy gap in this study shows it did overfit on this dataset, and Section IV-B shows it was not the best-scoring model. It is an ensemble of many decision trees. During training, each tree learns its own splitting rules by reducing Gini impurity (the scikit-learn default).

Instead of simple normal-versus-attack detection, the attack labels were grouped into three priority tiers:

- **Low Priority (Class 0):** This tier contains normal traffic, which poses no immediate threat; the training set contains 67,343 such instances.
- **Medium Priority (Class 1):** This tier contains probe (reconnaissance) attacks (e.g., `satan`, `ipsweep`, `nmap`, `portsweep`), 11,656 instances.
- **High Priority (Class 2):** This tier contains every attack that is not a probe: DoS attacks (e.g., `neptune`, `smurf`, `back`, `teardrop`) and remote-to-local and user-to-root attacks (e.g., `guess_passwd`, `buffer_overflow`), totaling 46,974 instances. In KDDTrain+ it is dominated by

TABLE I
PRIORITY TIER MAPPING AND CLASS COUNTS (KDDTRAIN+ ONLY)

Target Tier	Risk Level	Instances	Sample Attack Profiles (NSL-KDD)
Class 0	Low Priority	67,343	Normal Traffic, Benign Deviations
Class 1	Medium Priority	11,656	Reconnaissance, Probe (<code>satan</code> , <code>ipsweep</code> , <code>nmap</code>)
Class 2	High Priority	46,974	DoS, Root-level Compromise (<code>neptune</code> , <code>smurf</code> , <code>back</code>)

DoS records (approximately 45,927 of 46,974 instances), a figure consistent with the published NSL-KDD label distribution [1] rather than a value computed directly in this study’s pipeline.

The trees vote, and the majority class becomes the prediction. This was tested only on benchmark data, not live network traffic.

D. Implementation Notes

The model described in this study was implemented in Python using pandas 3.0.6, scikit-learn 1.9.1, XGBoost 3.4.1 and SHAP 0.52.0. All models used a fixed random seed (`random_state=42`). Random Forest and Decision Tree classifiers used scikit-learn default hyperparameters; XGBoost used `n_estimators=100` with `objective='multi:softprob'`. Five-fold stratified cross-validation also used `random_state=42`; the Gaussian noise stress test used a fixed NumPy seed (`np.random.seed(42)`).

IV. EXPERIMENTAL RESULTS AND DISCUSSION

A multi-part experimental evaluation process was conducted to assess the effectiveness, robustness, and reliability of the proposed prioritization framework. The evaluation had three parts: baseline performance, a feature-ablation test, and a Gaussian noise test. The entire experimental pipeline was evaluated using the official KDDTest+ file (22,544 records), which is fully independent of the training data.

A. Baseline Model Performance

Initially, the Random Forest ensemble model was trained and tested using the full feature space, without any external noise or modifications. Experimental results indicate that the proposed model achieved an overall accuracy of 74.80% on KDDTest+. Accuracy alone can hide differences between classes, so precision, recall and F1-score are also reported in Table II. Macro-averaged F1 was 0.73. Training accuracy on KDDTrain+ was 99.99%, substantially higher than the 74.80% test accuracy; this gap is discussed further in the Limitations section. Inference time for the trained Random Forest on the 22,544 KDDTest+ records was 0.5224 seconds (0.0232 ms per alert), measured once with `model.predict` on a single CPU-only machine with preprocessing excluded; this is a rough figure, not a real-time benchmark.

TABLE II
PER-CLASS RESULTS ON KDDTEST+

Priority Class	Precision	Recall	F1-Score	Accuracy
Class 0 (Low)	0.65	0.97	0.78	–
Class 1 (Medium)	0.85	0.60	0.70	–
Class 2 (High)	0.96	0.57	0.72	–
Macro Average	0.82	0.71	0.73	–
Overall System	–	–	–	74.80%

TABLE III
COMPARISON OF CLASSIFIERS ON KDDTEST+

ML Model	Test Acc.	Complexity	Interpretability
Decision Tree	76.47%	Low	High (single tree)
Random Forest	74.80%	Medium	Medium (SHAP)
XGBoost	77.58%	High	Medium (SHAP)

B. Comparative Model Evaluation

To validate the selection of our predictive engine, a comparative evaluation was conducted across three distinct machine learning architectures: an unpruned Decision Tree, our Random Forest ensemble, and an extreme gradient-boosted tree framework (XGBoost). The models were benchmarked on the same training set and the same KDDTest+ test set; Random Forest and Decision Tree used scikit-learn default hyperparameters, while XGBoost used `n_estimators=100`, so hyperparameter tuning effort was not identical across models.

In Table III, Complexity refers to the number of estimators used per model: 1 tree for Decision Tree, 100 trees for Random Forest, and 100 boosting rounds for XGBoost. As illustrated in Table III, the XGBoost architecture achieved the highest accuracy (77.58%), followed by the Decision Tree (76.47%), with the Random Forest ensemble scoring lowest (74.80%) on the held-out KDDTest+ set. Random Forest was used as the primary model for the SHAP and ablation analysis in this study, even though it scored lowest of the three on test accuracy. SHAP’s TreeExplainer also supports the Decision Tree and XGBoost models, so interpretability tooling alone does not justify this choice; XGBoost is the stronger candidate for future work. A very small search (`max_depth` of 10 or 20, `min_samples_leaf` fixed at 5, 50 trees, 2-fold CV on the training set) gave 73.89% test accuracy versus 74.80% for the default model. The search was too small to draw conclusions about tuning. Analysis of the classification report (Table II) shows uneven performance across classes, with notably higher recall for Low-priority alerts (0.97) than for High-priority alerts (0.57). Figure 2 shows the main weakness: 4,374 of 10,412 High-priority test alerts (42%) were labeled Low, and 814 of 2,421 Medium alerts (34%) were also labeled Low. For a prioritization system this is the most serious error type, since these alerts would sit at the bottom of an analyst’s queue. The model was evaluated on the fully independent KDDTest+ set, though this remains a static benchmark dataset and does not

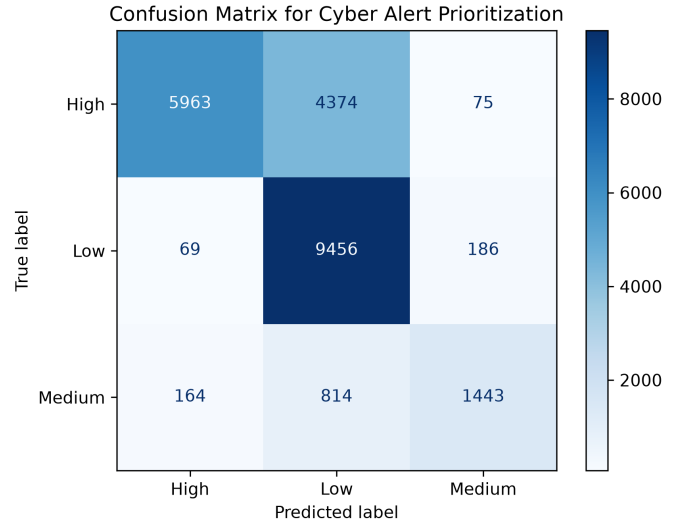


Fig. 2. Confusion matrix of the Random Forest on KDDTest+.

fully represent live, real-world network traffic.

C. Explainable AI (XAI) Analysis

Tree ensembles such as Random Forest are often treated as black boxes, and analysts may not trust a risk level without a reason. To look at this, the study uses SHAP (SHapley Additive exPlanations) values [5] and a plot of one decision tree from the forest. Together these show which features drive the model’s outputs; neither was tested with actual analysts. SHAP analysis was run on the Random Forest rather than XGBoost, since Random Forest was the model used for the ablation and noise tests in this study; extending SHAP to XGBoost is left for future work.

The global feature contributions obtained from SHAP, computed with `shap.TreeExplainer` on the first 500 rows of the test set (not a random sample) for computational efficiency, are shown in Figure 3. This summary plot ranks network features based on their average contribution across three threat tiers or classes (low, medium, and high priority). The plot shows that the two payload-size features, `src_bytes` (bytes sent from the source) and `dst_bytes` (bytes arriving at the destination), have the largest average impact on the model’s output. The connection-state flags `flag_SF` and `flag_S0` also appear among the top features. Fig. 3 shows only average importance, so it does not show how these flags push predictions toward any particular class.

SHAP values could give analysts a starting point for why an alert was ranked as it was, though any effect on triage time remains untested.

D. Feature Ablation Test

Attackers may try to change the traffic features a detector relies on. A feature ablation test was used as a rough proxy: a second Random Forest was trained and tested with four of the five most important features removed

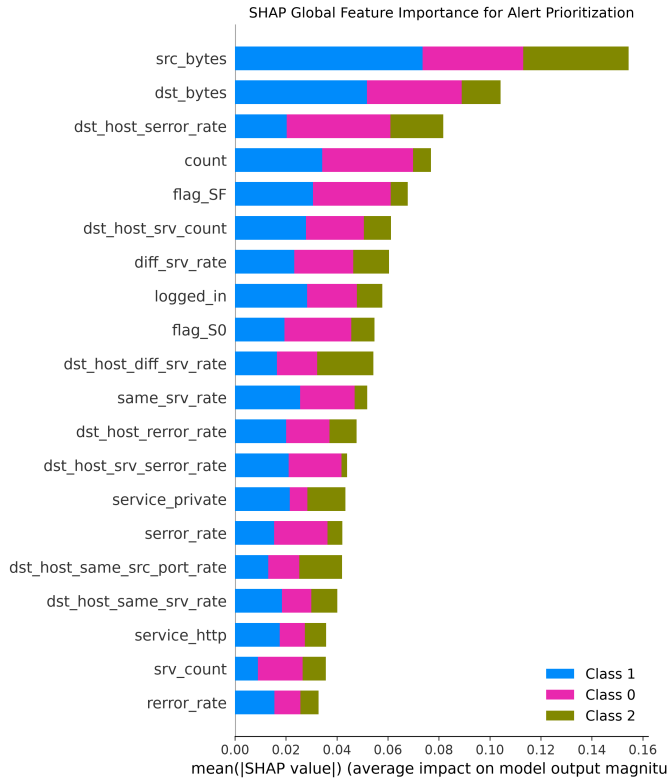


Fig. 3. SHAP global feature importance (mean |SHAP value|) for the Random Forest on the first 500 KDDTest+ rows. Legend classes follow alphabetical label order (Class 0 = High, Class 1 = Low, Class 2 = Medium), not the numbering in Table I.

(src_bytes, dst_bytes, dst_host_serror_rate, diff_srv_rate). Removing a feature is not the same as an attacker manipulating its value.

With those four features removed, the retrained model reached 74.19% accuracy, 0.61 percentage points below baseline. A likely reason is that NSL-KDD contains many correlated features (for example, several error-rate features), so other columns carry similar information. This is a single run, and it does not show that the model resists evasion by an adaptive attacker.

E. Noise Robustness Test

Real network measurements are noisier than benchmark records. A simple test was run by adding Gaussian noise to some test features and re-scoring the trained model. This is random noise, not an adversarial attack of the kind studied in [6]. During this process, Gaussian noise ($\mu = 0$, $\sigma = 0.1$) was injected into seven byte- and count-valued features (duration, src_bytes, dst_bytes, count, srv_count, dst_host_count, dst_host_srv_count); all other columns, including the 0–1 rate features and the one-hot columns, were left unmodified.

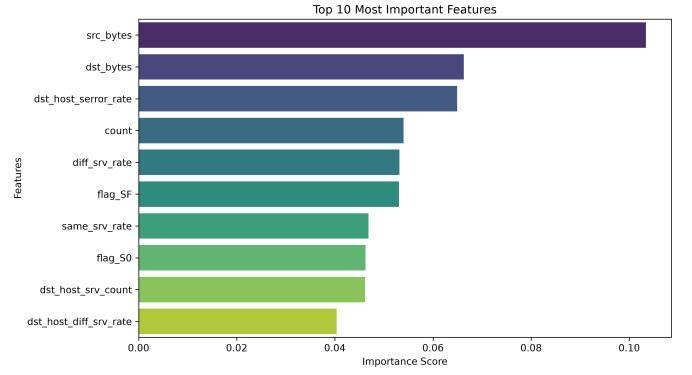


Fig. 4. Top 10 Random Forest feature importances (impurity-based, trained on KDDTrain+).

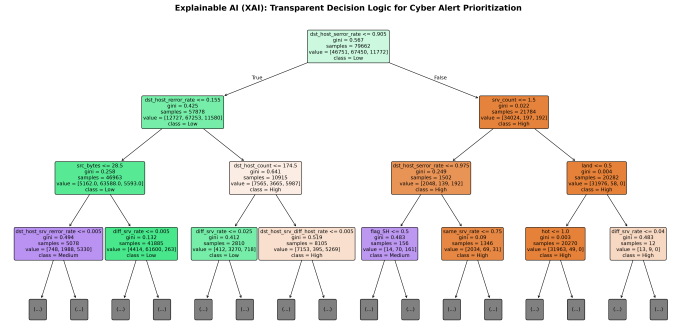


Fig. 5. First tree of the Random Forest, truncated to depth 3.

Accuracy stayed at 74.80% to two decimal places, matching the baseline. This is expected: these features range from 0 up to millions (bytes) or hundreds (counts), so noise with $\sigma = 0.1$ rarely moves a value across a tree split. The 0–1 rate features, where $\sigma = 0.1$ would matter, were not perturbed. The test therefore shows only insensitivity to very small noise; a fairer version would scale the noise to each feature’s standard deviation.

F. Decision Pathway Interpretation

This subsection looks at one decision tree from the forest.

Figure 5 shows the top three levels of the first of the forest’s 100 trees, using splits on several of the features shown to be important in Figure 4. Its leaves are labeled High, Low, or Medium. This is one tree out of 100, so it illustrates the kind of rule the forest uses; it does not explain any single prediction, which is a vote across all trees.

V. CONCLUSION

Large alert volumes and the resulting alert fatigue among analysts make it harder for SOCs to notice real attacks. This study trained a Random Forest on the NSL-KDD dataset to sort connection records into three priority levels: low, medium, and high.

The proposed model reached 74.80% test accuracy on KDDTest+, below the Decision Tree (76.47%) and XGBoost (77.58%), and recognized only 57% of High-priority alerts. In a feature-ablation test and a small-noise test, accuracy was 74.19% and 74.80%, respectively. SHAP values and an example decision tree show which features the model relies on; their usefulness to analysts was not tested.

VI. LIMITATIONS

Several limitations should be noted. First, training accuracy (99.99%) is substantially higher than test accuracy (74.80%), indicating the model overfits patterns specific to the training file that do not fully generalize to KDDTest+'s novel attack types. Second, five-fold stratified cross-validation on the training set produced a mean accuracy of 99.88% ($\pm 0.04\%$), which is far higher than the 74.80% accuracy observed on the independent KDDTest+ set; an exact-duplicate check on KDDTrain+ found 0 duplicate rows (0 when ignoring the 'level' column). A similarity check (cosine similarity > 0.99 , computed on unscaled features) on a 2,000-row sample found 119,008 near-duplicate pairs out of 1,999,000 (about 5.95%), suggesting many training records are very similar, which may inflate cross-validation scores. The main cause of the gap is probably that KDDTest+ has a different attack mix than KDDTrain+, including attack types absent from training; we did not test this. Cross-validation on the training file is therefore not a reliable estimate of test performance here. Third, the feature ablation and Gaussian noise experiments assess the model's sensitivity to feature removal and random noise, but do not simulate a deliberate, adaptive attacker attempting to evade the model, which would require a more targeted adversarial evaluation. Fourth, the SHAP-based explanations were computed on only the first 500 rows of KDDTest+, which may not be representative of the full test set, and SHAP values may be less reliable when input features are correlated, as can occur with network traffic statistics. Finally, all experiments were conducted on a static, offline benchmark dataset rather than live network traffic, so real-time performance has not yet been evaluated. NSL-KDD is derived from older traffic and does not include modern attack types. In addition, the priority tiers are our own grouping of NSL-KDD attack labels, not an established severity scale, and the records are labeled connections rather than real SOC alerts, so the study does not measure any reduction in analyst workload.

VII. FUTURE WORK

Although the current framework demonstrates promising results, there are several opportunities to further refine and enhance its practicality in future research:

- **Real-Time Streaming Data:** The model could be connected to live network interfaces or streaming pipelines to test it on live traffic.
- **Deep Learning Models:** Models such as CNN-LSTM could be compared against the tree-based models.

- **Threat Intelligence Feeds:** A threat intelligence feed could be added as an extra input; whether it helps with new attacks was not tested.
- **Reducing the Overfitting Gap:** Future work will investigate hyperparameter tuning (e.g., limiting tree depth) and further inspection of near-duplicate records within KDDTrain+ to narrow the gap between training and test accuracy.
- **Fair Hyperparameter Comparison:** The small grid search on Random Forest (two values of max depth, one value of min samples leaf) did not improve accuracy; future work should run a broader search and apply comparable tuning to Decision Tree and XGBoost.
- **Newer Dataset:** The model should be tested on a more recent intrusion dataset such as CIC-IDS2017.

Overall, this is a small benchmark study, and its results are a starting point for further work on alert prioritization.

REFERENCES

- [1] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," in *Proc. 2009 IEEE Symp. Computational Intelligence for Security and Defense Applications (CISDA)*, Ottawa, ON, Canada, 2009, pp. 1–6.
- [2] M. Wang, K. Zheng, Y. Yang, and X. Wang, "An Explainable Machine Learning Framework for Intrusion Detection Systems," *IEEE Access*, vol. 8, pp. 73127–73141, 2020, doi: 10.1109/ACCESS.2020.2988359.
- [3] D. Gaspar, P. Silva, and C. Silva, "Explainable AI for Intrusion Detection Systems: LIME and SHAP Applicability on Multi-Layer Perceptron," *IEEE Access*, vol. 12, pp. 30164–30175, 2024, doi: 10.1109/ACCESS.2024.3368377.
- [4] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [5] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, Long Beach, CA, USA, 2017, pp. 4765–4774.
- [6] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and Harnessing Adversarial Examples," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.